

**UNIVERSIDAD PRIVADA ANTENOR ORREGO
FACULTAD DE INGENIERÍA
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS E INTELIGENCIA
ARTIFICIAL**



**Modelo Predictivo de Riesgo de Morosidad en Clientes Peruanos
Usando Aprendizaje Supervisado**

Curso:

Aprendizaje Estadístico

Docente:

Sagastegui Chigne, Teobaldo Hernan.

Estudiantes:

Rodriguez Correa, Franck Williams

Gutierrez Cossa, Cesar Alberto

Ticlla Silva, Abel Gerardo

INDICE

I. Introducción	5
1.1 Título del Proyecto	5
1.2 Antecedentes	5
1.3 Problema a Resolver	5
1.4 Objetivos	5
II. Requerimientos	7
2.1 Definición del Dominio	7
2.2 Determinación de Requisitos	7
III. Planteamiento del Dataset, Pre-Procesamiento y Normalización	9
3.1 Medidas, Datos y Bases de Datos para la Elaboración del Dataset	9
3.2 Normalización y/o Filtrado de Datos	9
3.3 Planteamiento de los Conjuntos de Datos	10
IV. Aprendizaje	11
4.1 Planteamiento del Modelo de Aprendizaje	11
4.2 Desarrollo e Implementación del Modelo	11
V. Comprobación	12
5.1 Entrenamiento del Modelo — Uso del Dataset de Entrenamiento y de Prueba	12
VI. Evaluación	14
6.1 Métricas y Evaluación del Modelo	14
6.2 Evaluación de Desempeño del Sistema de Aprendizaje Estadístico	14
VII. Despliegue (Deploy) del Sistema del Proyecto de Aprendizaje Estadístico	15
7.1 Publicación del Proyecto en GitHub	15
7.2 Deploy del Sistema de Predicción	15
Referencias Bibliográficas	16

I. Introducción

1.1 Título del Proyecto

Modelo Predictivo de Riesgo de Morosidad en Clientes Peruanos usando Aprendizaje Supervisado

1.2 Antecedentes

Cabe señalar que gran parte de los modelos modernos de gestión de riesgo crediticio utilizados en Latinoamérica, incluido el sistema financiero peruano, toman como referencia metodologías implementadas en entidades bancarias europeas y estándares internacionales de supervisión financiera, especialmente aquellos relacionados con evaluación crediticia, scoring financiero y control de morosidad. En ese sentido, los estudios desarrollados en España y Europa constituyen antecedentes relevantes para comprender la evolución y aplicación de modelos predictivos orientados a la reducción del riesgo crediticio en el contexto peruano

En España, Universidad de Zaragoza desarrolló la investigación denominada “Morosidad en las entidades financieras”, en la cual se analizaron los factores económicos y financieros asociados al incremento de la morosidad bancaria. El estudio concluyó que variables como el desempleo, el sobreendeudamiento y las crisis económicas incrementan significativamente el riesgo de incumplimiento de pago, afectando directamente la estabilidad y rentabilidad de las entidades financieras. Asimismo, se destaca la importancia de fortalecer los mecanismos de evaluación crediticia y control de riesgo dentro del sistema bancario.

De igual manera, Universidad de Valladolid realizó el estudio “Gestión de riesgos en las entidades financieras: el riesgo de crédito y morosidad”, cuyo objetivo fue analizar el impacto del riesgo crediticio sobre la estabilidad financiera de las entidades bancarias españolas. La investigación evidenció que la morosidad constituye uno de los principales factores de deterioro financiero y que una adecuada gestión del riesgo crediticio permite reducir pérdidas económicas y mejorar la calidad de cartera de las instituciones financieras.

Por otro lado, Fundación de las Cajas de Ahorros publicó el estudio “Análisis sectorial de la morosidad bancaria: España en el contexto europeo”, donde se evaluó el comportamiento de la morosidad bancaria en distintos sectores económicos. Los resultados demostraron que determinados sectores presentan mayor vulnerabilidad financiera y mayores niveles de incumplimiento crediticio, resaltando la necesidad de implementar herramientas de monitoreo y análisis predictivo que permitan anticipar escenarios de riesgo financiero.

Asimismo, la Revista Universitaria Europea publicó el artículo científico “El riesgo de crédito en los principales bancos españoles en tiempos del Covid-19”, en el cual se analizaron entidades financieras como BBVA, Banco Santander y CaixaBank durante el periodo de pandemia. El estudio concluyó que las crisis económicas incrementan considerablemente la exposición al riesgo crediticio, generando la necesidad de fortalecer los sistemas de clasificación y evaluación financiera mediante modelos predictivos y técnicas estadísticas.

Management Solutions publicó el estudio *“La gestión de la morosidad en entidades financieras”*, en el cual se analiza la importancia de implementar modelos predictivos y herramientas analíticas para la identificación temprana de clientes con alto riesgo de incumplimiento financiero. El estudio señala que la segmentación de clientes, el scoring crediticio y el análisis estadístico permiten optimizar la gestión de cobranza y reducir el riesgo operativo dentro de las entidades bancarias. Asimismo, concluye que el uso de modelos predictivos mejora la toma de decisiones financieras y fortalece la estabilidad de la cartera crediticia. Este antecedente aporta al presente proyecto debido a que respalda la utilización de técnicas de aprendizaje estadístico para la predicción de morosidad en el sistema financiero.

En el contexto peruano, la calidad de cartera y los niveles de morosidad representan indicadores determinantes para la evaluación del riesgo crediticio. La Superintendencia de Banca, Seguros y AFP (SBS) publica periódicamente información estadística sobre morosidad por tipo de crédito y modalidad, permitiendo analizar el comportamiento de pago dentro del sistema financiero formal. Esta información constituye una base relevante para el desarrollo de modelos predictivos orientados a anticipar escenarios de incumplimiento y fortalecer la gestión de riesgo crediticio.

En el contexto nacional, Universidad Nacional del Centro del Perú desarrolló la investigación *“Gestión de riesgo de crédito y la morosidad en la Región Centro de Mibanco S.A.”*, orientada a determinar la relación entre la gestión del riesgo crediticio y los niveles de morosidad en clientes del sistema financiero peruano. Los resultados evidenciaron que variables como historial crediticio, capacidad de pago, endeudamiento y clasificación financiera influyen significativamente en el incumplimiento de obligaciones crediticias. Además, la investigación concluyó que una adecuada evaluación del riesgo permite reducir la cartera morosa y mejorar la estabilidad financiera de las entidades crediticias.

Asimismo, la evaluación crediticia en el sistema financiero peruano incorpora variables asociadas al comportamiento histórico de pago, nivel de endeudamiento, capacidad de pago y clasificación de riesgo del deudor. La clasificación crediticia consolidada por la Superintendencia de Banca, Seguros y AFP constituye un insumo relevante para comprender la exposición al riesgo dentro del sistema bancario, permitiendo construir perfiles financieros útiles para el modelamiento predictivo de morosidad.

1.3 Problema a Resolver

¿Es posible predecir, con base en el perfil socioeconómico e historial crediticio de un cliente, si este incurrirá en morosidad ($\text{mora} = 1$) o pagará puntualmente ($\text{mora} = 0$) sus obligaciones financieras?

Actualmente, la evaluación crediticia depende en gran parte de criterios subjetivos o análisis manuales que no aprovechan el potencial predictivo de los datos históricos. Esto genera dos problemas principales:

- Riesgo financiero elevado: se otorgan préstamos a clientes con alta probabilidad de mora.
- Exclusión financiera: se niegan préstamos a clientes con buen perfil de pago por criterios de evaluación imprecisos.

1.4 Objetivos

Objetivo General:

Desarrollar un modelo de clasificación binaria que prediga la morosidad de un cliente bancario en el sistema financiero peruano, utilizando técnicas de Aprendizaje Supervisado implementadas en Google Colab y validadas con Weka.

Objetivos Específicos:

- Analizar y preprocesar el dataset BankDefaultAnalysis garantizando la calidad de los datos, apoyándose en Weka para la exploración estadística inicial.
- Implementar y comparar modelos de clasificación (Regresión Logística, Árbol de Decisión, Random Forest) en Google Colab usando Python y scikit-learn.
- Evaluar el desempeño de los modelos mediante métricas estándar: Accuracy, Precisión, Recall, F1-Score y AUC-ROC.
- Comparar los resultados obtenidos en Google Colab con los resultados de Weka para validar la consistencia del modelo seleccionado.

II. Requerimientos

2.1 Definición del Dominio

El presente proyecto se desarrolla dentro del dominio de las Finanzas y la Gestión del Riesgo Crediticio. Específicamente, se enfoca en el área de *scoring crediticio*, cuyo propósito es estimar la probabilidad de incumplimiento de obligaciones financieras por parte de un cliente dentro del sistema financiero peruano.

Desde la perspectiva del aprendizaje automático, el problema se modela como una tarea de clasificación binaria supervisada, donde el sistema debe predecir si un cliente presenta riesgo de morosidad en función de variables socioeconómicas, laborales y crediticias.

La variable objetivo del modelo corresponde a la condición de morosidad del cliente y se representa de la siguiente manera:

- mora = 0: Cliente paga puntualmente (no es moroso). ✓
- mora = 1: Cliente se atrasa en sus pagos (es moroso). ✗

Variables del dataset:

El presente proyecto utiliza el dataset denominado “*BankDefaultAnalysis — Morosidad del Sistema Financiero Peruano*”, publicado por Calderón Baldeón (2021) en la plataforma Kaggle. Dicho conjunto de datos contiene información asociada al comportamiento crediticio de clientes del sistema financiero peruano, incluyendo variables socioeconómicas, financieras y relacionadas con riesgo de morosidad.

Variable	Tipo	Descripción
mora	Binaria (0/1)	Variable objetivo: 0 = paga al día, 1 = cliente moroso
atraso	Numérica	Días de atraso histórico del cliente (0–245 días)
vivienda	Categórica	Tipo de vivienda (Familiar, Propia, Alquilada, etc.)
edad	Numérica	Edad del cliente en años (20–85)
dias_lab	Numérica	Días laborados en su empleo actual (2956–20700)
exp_sf	Numérica	Meses de experiencia en el sistema financiero
nivel_ahorro	Numérica	Nivel de ahorro del cliente (0 a 12)
ingreso	Numérica	Ingreso mensual del cliente en S/.
linea_sf	Numérica	Línea de crédito en el sistema financiero
deuda_sf	Numérica	Deuda actual en el sistema financiero
score	Numérica	Score de calificación crediticia (134–266)
zona	Categórica	Región geográfica del cliente
clasif_sbs	Numérica	Clasificación SBS (0=normal, 4=pérdida)
nivel_educ	Categórica	Nivel de educación del cliente

2.2 Determinación de Requisitos

Para el desarrollo del modelo predictivo se requiere un conjunto de datos representativo del comportamiento crediticio de clientes pertenecientes al sistema financiero peruano. El dataset utilizado contiene 8,399 registros y 14 variables asociadas a características socioeconómicas, laborales y financieras de los clientes.

Asimismo, la variable objetivo (*mora*) debe encontrarse correctamente etiquetada de forma binaria para permitir el entrenamiento supervisado de los modelos de clasificación. Las variables predictoras deben proporcionar información suficiente para identificar patrones asociados al incumplimiento financiero y al riesgo crediticio.

Adicionalmente, el dataset debe permitir la aplicación de procesos de preprocesamiento, normalización, imputación de valores faltantes y balanceo de clases mediante técnicas de aprendizaje estadístico.

Requisitos de Datos:

- Dataset con 8,399 registros de clientes con historial crediticio real del sistema financiero peruano.
- Variable objetivo (*mora*) claramente etiquetada de forma binaria.
- Variables predictoras que cubren aspectos socioeconómicos, laborales y crediticios.

Requisitos del Modelo:

- El modelo debe clasificar clientes como morosos (1) o no morosos (0).
- Accuracy mínima esperada: 75% en el conjunto de prueba.
- AUC-ROC mínimo esperado: 0.80.

Requisitos del Sistema:

- Entorno de exploración estadística: Weka (con archivo .arff).
- Entorno de desarrollo principal: Google Colab (Python 3.x).
- Bibliotecas Python: scikit-learn, pandas, numpy, matplotlib, seaborn, imbalanced-learn.
- Repositorio: GitHub con documentación completa y notebook ejecutable.

III. Planteamiento del Dataset, Pre-Procesamiento y Normalización

3.1 Medidas, Datos y Bases de Datos para la Elaboración del Dataset

Fuente del Dataset:

- Nombre: BankDefaultAnalysis — Morosidad del Sistema Financiero
- Autor: Luis Humberto Calderón Baldeón
- Plataforma: Kaggle (kaggle.com/datasets/luishcaldernb/morosidad)
- Archivo: data.csv
- Dimensiones: 14 columnas × 8,399 registros
- Licencia: Uso académico permitido

2. Carga del Dataset

```
from google.colab import files
uploaded = files.upload()

import io
df = pd.read_csv(io.BytesIO(list(uploaded.values())[0]))

print(f"Dataset cargado: {df.shape[0]} filas x {df.shape[1]} columnas")
df.head(10)
```

Elegir archivos Sin archivos seleccionados Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving data.csv to data.csv

Dataset cargado: 8399 filas x 14 columnas

	mora	atraso	vivienda	edad	dias_lab	exp_sf	nivel_ahorro	ingreso	linea_sf	deuda_sf	score	zona	clasif_sbs	nivel_educ
0	0	235	FAMILIAR	30	3748	93.0	5	3500.00	NaN	0.00	214	Lima	4	UNIVERSITARIA
1	0	18	FAMILIAR	32	4598	9.0	12	900.00	1824.67	1933.75	175	La Libertad	1	TECNICA
2	0	0	FAMILIAR	26	5148	8.0	2	2400.00	2797.38	188.29	187	Lima	0	UNIVERSITARIA
3	0	0	FAMILIAR	36	5179	20.0	12	2700.00	NaN	0.00	187	Ancash	0	TECNICA
4	0	0	FAMILIAR	46	3960	NaN	1	3100.00	2000.00	11010.65	189	Lima	0	TECNICA
5	0	22	FAMILIAR	25	4874	9.0	12	2200.00	449.92	496.58	220	Lima	0	UNIVERSITARIA
6	0	9	FAMILIAR	30	3930	12.0	8	2100.00	4827.64	850.21	193	Lima	0	UNIVERSITARIA
7	0	8	PROPIA	55	4995	23.0	12	5593.21	10467.00	18620.80	199	Piura	0	TECNICA
8	0	2	FAMILIAR	30	5026	NaN	0	2000.00	7315.41	0.00	156	Piura	0	UNIVERSITARIA
9	0	0	FAMILIAR	31	4964	NaN	4	3800.00	NaN	0.00	192	Ica	0	TECNICA

Distribución de la variable objetivo:

Clase	Cantidad	Porcentaje
No Moroso (mora = 0)	2,484	29.6%
Moroso (mora = 1)	5,915	70.4%
TOTAL	8,399	100%

Nota: El dataset presenta desbalance de clases (70.4% morosos vs. 29.6% no morosos), lo cual fue atendido mediante la técnica SMOTE durante el preprocesamiento en Google Colab.

Valores Nulos detectados:

3.2 Valores nulos por columna

```
[ ]  
  
# Análisis de valores nulos  
nulos = df.isnull().sum()  
porc_nulos = (nulos / len(df)) * 100  
  
resumen_nulos = pd.DataFrame({  
    'Valores Nulos': nulos,  
    'Porcentaje (%)': porc_nulos.round(2)  
}).sort_values('Porcentaje (%)', ascending=False)  
  
print("VALORES NULOS POR COLUMNA:")  
print(resumen_nulos[resumen_nulos['Valores Nulos'] > 0])
```

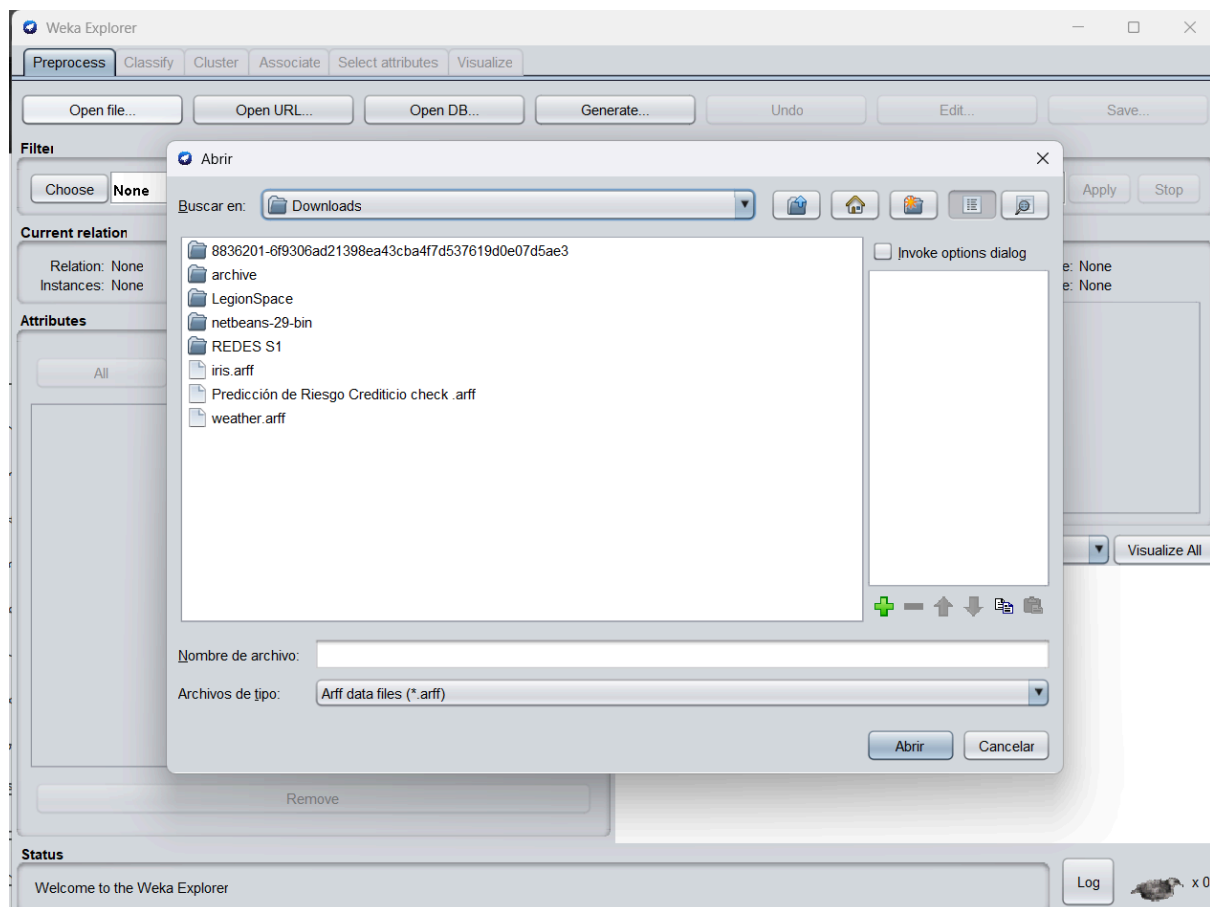
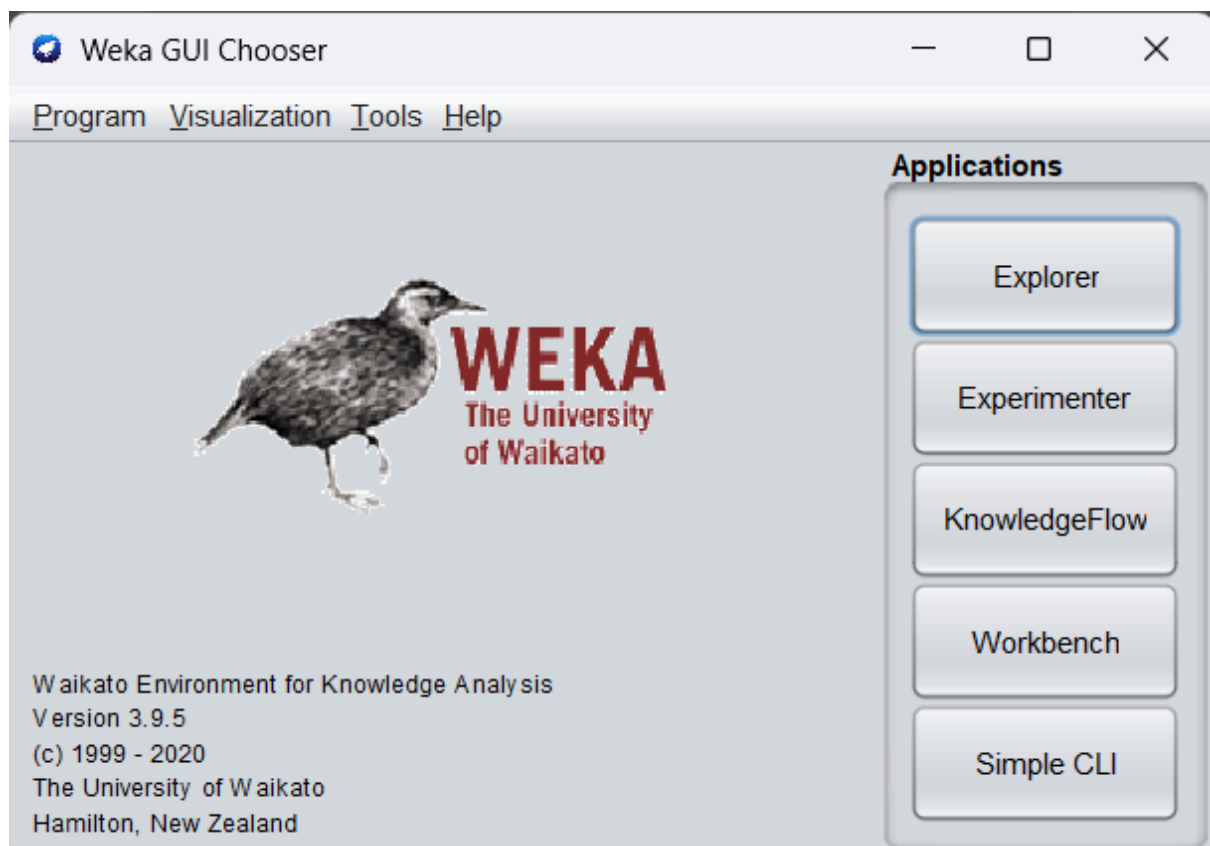
```
▼  
VALORES NULOS POR COLUMNA:  
      Valores Nulos  Porcentaje (%)  
exp_sf           1830           21.79  
linea_sf          1127           13.42  
deuda_sf           461            5.49
```

Variable	Valores Nulos	Porcentaje
exp_sf	1,830	21.79%
linea_sf	1,127	13.42%
deuda_sf	461	5.49%
Resto de variables	0	0%

Exploración Inicial con Weka:

Previo al preprocesamiento en Google Colab, se utilizó Weka como herramienta de exploración estadística inicial. El archivo .arff fue generado a partir del dataset original y cargado en Weka Explorer para:

- Visualizar la distribución de la variable objetivo (mora) e identificar el balance de clases.
- Analizar estadísticas descriptivas de cada variable (media, desviación estándar, mínimos y máximos).
- Identificar valores faltantes en las columnas exp_sf, linea_sf y deuda_sf.
- Obtener una primera aproximación al poder predictivo de cada variable.



3.2 Normalización y/o Filtrado de Datos

Se aplicaron los siguientes pasos de preprocesamiento en Google Colab:

- Tratamiento de valores nulos: imputación con la mediana para variables numéricas (exp_sf, linea_sf, deuda_sf).
- Codificación de variables categóricas: aplicación de One-Hot Encoding para las variables vivienda, zona y nivel_educ.
- Normalización de variables numéricas: aplicación de StandardScaler (media=0, desviación=1) a todas las variables continuas.
- Detección y balanceo de clases: dado el desbalance detectado (70.4% morosos vs. 29.6% no morosos), se aplicó la técnica SMOTE (Synthetic Minority Over-sampling Technique) sobre el conjunto de entrenamiento.

4. Preprocesamiento y Normalización de Datos

4.1 Tratamiento de valores nulos

```
[1] # Separar variables numéricas y categóricas
vars_categoricas = df.select_dtypes(include=['object']).columns.tolist()
vars_numericas_todas = df.select_dtypes(include=[np.number]).columns.tolist()
vars_numericas_features = [v for v in vars_numericas_todas if v != 'mora']

print(f"Variables categóricas: {vars_categoricas}")
print(f"Variables numéricas (features): {vars_numericas_features}")

# Imputar nulos: mediana para numéricas, moda para categóricas
for col in vars_numericas_features:
    if df[col].isnull().sum() > 0:
        mediana = df[col].median()
        df[col].fillna(mediana, inplace=True)
        print(f"    {col}: imputado con mediana ({mediana:.2f})")

for col in vars_categoricas:
    if df[col].isnull().sum() > 0:
        moda = df[col].mode()[0]
        df[col].fillna(moda, inplace=True)
        print(f"    {col}: imputado con moda ({moda})")

print(f"\nValores nulos restantes: {df.isnull().sum().sum()}")

Variables categóricas: ['vivienda', 'zona', 'nivel_educ']
Variables numéricas (features): ['atraso', 'edad', 'dias_lab', 'exp_sf', 'nivel_ahorro', 'ingreso', 'linea_sf', 'deuda_sf', 'score', 'clasif_sbs']
exp_sf: imputado con mediana (20.00)
linea_sf: imputado con mediana (4030.12)
deuda_sf: imputado con mediana (2258.76)

Valores nulos restantes: 0
```

4.2 Codificación de variables categóricas (One-Hot Encoding)

```
[ ] ▶ # One-Hot Encoding para variables categóricas
df_encoded = pd.get_dummies(df, columns=vars_categoricas, drop_first=False)

print(f"Dimensiones originales: {df.shape}")
print(f"Dimensiones tras encoding: {df_encoded.shape}")
print(f"\nColumnas nuevas generadas:")
nuevas_cols = [c for c in df_encoded.columns if c not in df.columns]
for c in nuevas_cols:
    print(f" - {c}")
```

```
▼ ... Dimensiones originales: (8399, 14)
Dimensiones tras encoding: (8399, 43)
```

Columnas nuevas generadas:

- vivienda_ALQUILADA
- vivienda_FAMILIAR
- vivienda_PROPIA
- zona_Amazonas
- zona_Ancash
- zona_Apurimac
- zona_Arequipa
- zona_Ayacucho
- zona_Cajamarca
- zona_Callao
- zona_Cuzco
- zona_Huancavelica
- zona_Huanuco
- zona_Ica
- zona_Junin
- zona_La Libertad
- zona_Lambayeque
- zona_Lima
- zona_Loreto
- zona_Madre de Dios
- zona_Moquegua
- zona_Pasco
- zona_Piura

4.3 Separación de features y target

```
[ ] ▶ # Separar X (features) e y (target)
X = df_encoded.drop('mora', axis=1)
y = df_encoded['mora']

print(f"Shape de X (features): {X.shape}")
print(f"Shape de y (target): {y.shape}")
print(f"\nDistribución de clases en y:")
print(y.value_counts())
```

```
▼ ... Shape de X (features): (8399, 42)
Shape de y (target): (8399,)

Distribución de clases en y:
mora
1    5915
0    2484
Name: count, dtype: int64
```

4.4 División del dataset (70% train / 15% test / 15% validación)

```
[ ] # División estratificada del dataset
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.30, random_state=42, stratify=y
)

X_test, X_val, y_test, y_val = train_test_split(
    X_temp, y_temp, test_size=0.50, random_state=42, stratify=y_temp
)

print(" División del Dataset:")
print(f" Entrenamiento (Train): {X_train.shape[0]} registros ({X_train.shape[0]/len(X)*100:.1f}%)")
print(f" Prueba (Test): {X_test.shape[0]} registros ({X_test.shape[0]/len(X)*100:.1f}%)")
print(f" Validación (Val): {X_val.shape[0]} registros ({X_val.shape[0]/len(X)*100:.1f}%)")
```

```
▼ División del Dataset:
Entrenamiento (Train): 5879 registros (70.0%)
Prueba (Test): 1260 registros (15.0%)
Validación (Val): 1260 registros (15.0%)
```

4.5 Normalización (StandardScaler)

```
[ ]  
# Normalización con StandardScaler  
scaler = StandardScaler()  
  
X_train_scaled = scaler.fit_transform(X_train)  
X_test_scaled  = scaler.transform(X_test)  
X_val_scaled   = scaler.transform(X_val)  
  
print(" Normalización aplicada (media=0, desviación estándar=1)")  
print(f"Media de una feature antes: {X_train.iloc[:, 0].mean():.4f}")  
print(f"Media de una feature después: {X_train_scaled[:, 0].mean():.4f}")
```

Normalización aplicada (media=0, desviación estándar=1)
Media de una feature antes: 4.5970
Media de una feature después: 0.0000

4.6 Balanceo de clases con SMOTE (si aplica)

```
[ ]  
# Verificar desbalance y aplicar SMOTE si es necesario  
ratio = y_train.value_counts()  
desbalance = ratio.min() / ratio.max()  
  
print(f"Distribución de clases en Train:")  
print(ratio)  
print(f"Ratio de desbalance: {desbalance:.2f}")  
  
if desbalance < 0.7:  
    print("\n Desbalance detectado – Aplicando SMOTE...")  
    smote = SMOTE(random_state=42)  
    X_train_scaled, y_train = smote.fit_resample(X_train_scaled, y_train)  
    print(f" Clases balanceadas: {pd.Series(y_train).value_counts().to_dict()}")  
else:  
    print("\n Dataset balanceado – No se aplica SMOTE")
```

... Distribución de clases en Train:
mora
1 4140
0 1739
Name: count, dtype: int64
Ratio de desbalance: 0.42

Desbalance detectado – Aplicando SMOTE...
Clases balanceadas: {0: 4140, 1: 4140}

En Weka, se aplicaron los filtros ReplaceMissingValues y NumericToNominal para compatibilizar el dataset con los clasificadores evaluados, tal como se observa en la configuración de los modelos ejecutados.

Aunque el dataset no presentaba valores nulos en la variable objetivo, se aplicaron los filtros como parte del proceso estándar de preparación de datos.

Weka Explorer

Preprocess Classify Cluster Associate Select attributes Visualize

Open file... Open URL... Open DB... Generate... Undo Edit... Save...

Filter

weka

- filters
 - AllFilter
 - MultiFilter
 - RenameRelation
- supervised
- unsupervised
 - attribute
 - Add
 - AddCluster
 - AddExpression
 - AddID
 - AddNoise
 - AddUserFields
 - AddValues
 - CartesianProduct
 - Center
 - ChangeDateFormat
 - ClassAssigner
 - ClusterMembership
 - Copy
 - DateToNumeric
 - Discretize
 - FirstOrder
 - FixedDictionaryStringToWordVector
 - InterquartileRange

Selected attribute

Name: mora
Missing: 0 (0%)
Distinct: 2
Type: Nominal
Unique: 0 (0%)

No.	Label	Count	Weight
1	0	2484	2484.0
2	1	5915	5915.0

Class: nivel_educ (Nom)

Visualize All

2484

5915

Filter:

Choose
ReplaceMissingValues
Apply
Stop

Current relation

Relation: data-weka.filters.unsuperv...
Instances: 8399
Attributes: 14
Sum of weights: 8399

Attributes

All
None
Invert
Pattern

No.	Name
1	☒ mora
2	☐ atraso
3	☐ vivienda
4	☐ edad
5	☐ dias_lab
6	☐ exp_sf
7	☐ nivel_ahorro
8	☐ ingreso
9	☐ linea_sf
10	☐ deuda_sf
11	☐ score
12	☐ zona
13	☐ clasif_sbs
14	☐ nivel_educ

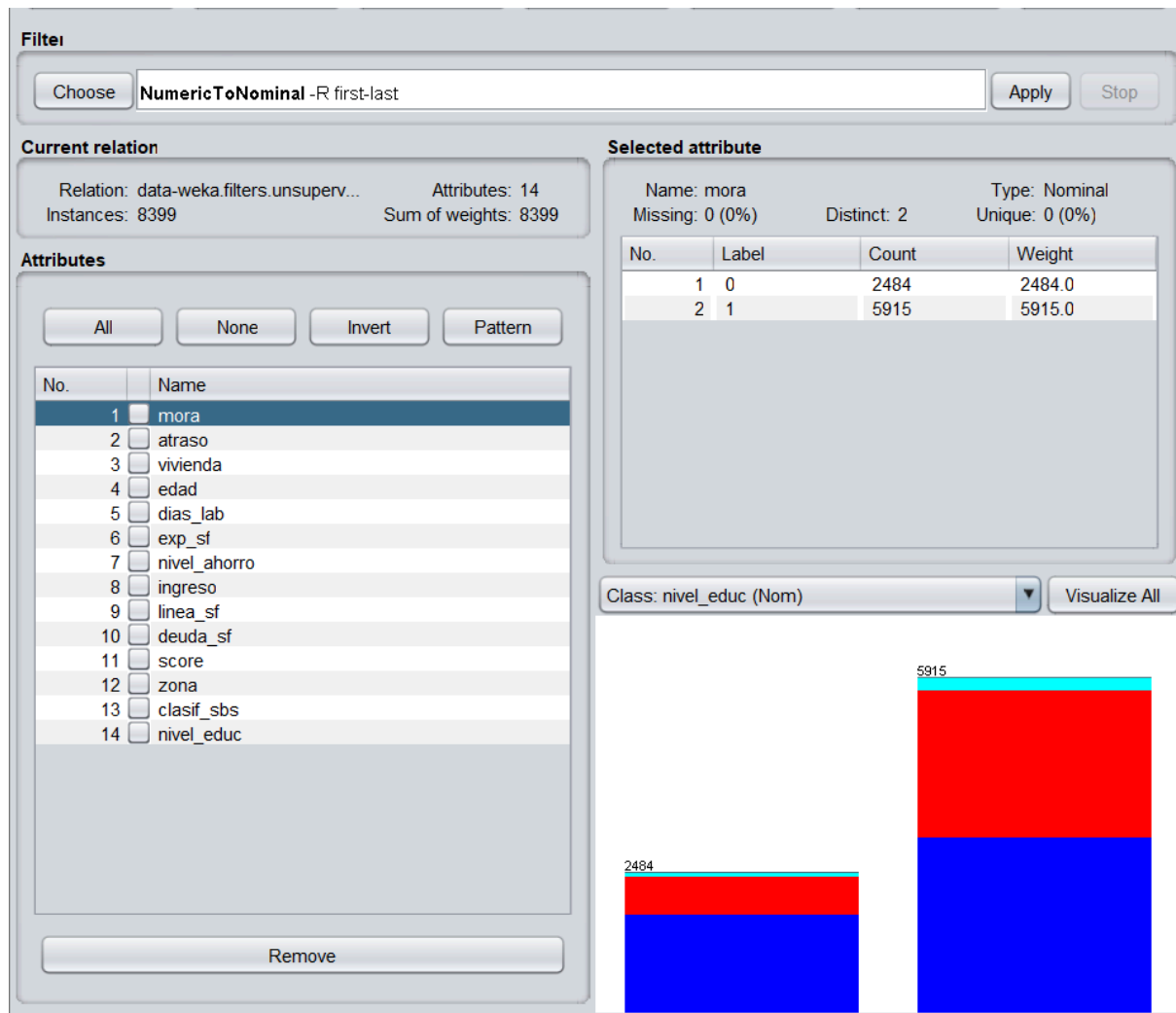
Remove

Selected attribute

Name: mora
Missing: 0 (0%)
Distinct: 2
Type: Nominal
Unique: 0 (0%)

No.	Label	Count	Weight
1	0	2484	2484.0
2	1	5915	5915.0

Class: nivel_educ (Nom)
Visualize All



3.3 Planteamiento de los Conjuntos de Datos

4.4 División del dataset (70% train / 15% test / 15% validación)

```
# División estratificada del dataset
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.30, random_state=42, stratify=y
)

X_test, X_val, y_test, y_val = train_test_split(
    X_temp, y_temp, test_size=0.50, random_state=42, stratify=y_temp
)

print(" División del Dataset:")
print(f" Entrenamiento (Train): {X_train.shape[0]} registros ({X_train.shape[0]/len(X)*100:.1f}%)")
print(f" Prueba (Test): {X_test.shape[0]} registros ({X_test.shape[0]/len(X)*100:.1f}%)")
print(f" Validación (Val): {X_val.shape[0]} registros ({X_val.shape[0]/len(X)*100:.1f}%)")
```

División del Dataset:

Entrenamiento (Train): 5879 registros (70.0%)

Prueba (Test): 1260 registros (15.0%)

Validación (Val): 1260 registros (15.0%)

Conjunto	Proporción	Uso
Entrenamiento (Training)	70%	Ajuste de parámetros del modelo
Prueba (Test)	15%	Evaluación del rendimiento final del modelo
Validación (Cross-Validation)	15% / k=5	Selección de hiperparámetros mediante k-Fold Estratificado

Se usó Stratified Split tanto en Google Colab como en Weka (Percentage Split 70%) para mantener la proporción de clases en cada subconjunto. Adicionalmente, se realizó validación cruzada estratificada con k=5 pliegues en ambas herramientas para garantizar la estabilidad de los resultados.

IV. Aprendizaje

4.1 Planteamiento del Modelo de Aprendizaje

Tipo de aprendizaje: Supervisado — Clasificación Binaria.

Se plantea comparar los siguientes modelos, implementados en Google Colab con la biblioteca scikit-learn y también ejecutados en Weka para validar la consistencia de los resultados:

Modelo	Tipo	Justificación
Regresión Lineal (adaptada)	Regresión/Clasificación	Permite explorar la relación lineal entre las variables y la mora. Se usa como punto de partida para entender el comportamiento del dataset.
Regresión Logística	Clasificación	Modelo base (baseline), interpretable y eficiente para clasificación binaria. Permite obtener probabilidades de mora directamente. (Weka: functions.Logistic)
Árbol de Decisión (J48)	Clasificación	Permite visualizar las reglas de decisión crediticia de forma intuitiva y comprensible. (Weka: trees.J48 -C 0.25 -M 2)
Random Forest	Clasificación (Ensemble)	Modelo robusto que combina múltiples árboles (100 iteraciones). Maneja bien datos desbalanceados y alta dimensionalidad. (Weka: trees.RandomForest -P 100 -I 100)

El modelo final fue seleccionado en base al mejor AUC-ROC y F1-Score en el conjunto de validación, siendo Random Forest el ganador en ambas herramientas (Weka y Google Colab).

4.2 Desarrollo e Implementación del Modelo

El desarrollo siguió el siguiente pipeline en Google Colab (Python):

- Carga y exploración inicial del dataset (EDA): análisis de distribuciones, correlaciones y valores nulos.
- Exploración estadística inicial en Weka: carga del archivo .arff, visualización de distribuciones y análisis de atributos.
- Preprocesamiento: imputación de nulos, One-Hot Encoding, StandardScaler y SMOTE para balanceo de clases.
- División estratificada del dataset: 70% entrenamiento, 15% prueba, 15% validación.
- Entrenamiento de los modelos candidatos con los datos de entrenamiento.

- Validación cruzada k-Fold (k=5) sobre el conjunto de entrenamiento.
- Ejecución paralela de los modelos en Weka (Percentage Split 70% y Cross-Validation 5-fold) para comparación y validación de resultados.
- Selección del modelo final (Random Forest) en base a métricas de evaluación.
- Exportación del modelo mediante joblib (.pkl) para su posterior despliegue.

Weka Explorer

Preprocess Classify Cluster Associate Select attributes Visualize

Classifier

Choose **RandomForest** -P 100 -I 100 -num-slots 1 -K 0 -M 1.0 -V 0.001 -S 1

Test options

☐ Use training set
☐ Supplied test set
☐ Cross-validation Folds 10
☒ Percentage split % 70

(Nom) mora

Result list (right-click for options)

00:48:48 - functions.Logistic
 00:49:40 - trees.J48
 00:50:30 - trees.RandomForest

Classifier output

```

=== Run information ===

Scheme:      weka.classifiers.trees.RandomForest -P 100 -I 100 -num-slots 1 -K 0 -M 1.0 -V 0.001 -S 1
Relation:    data-weka.filters.unsupervised.attribute.ReplaceMissingValues-weka.filters.unsupervised.attribute.NumericToNominal-RI
Instances:   8399
Attributes:  14
             mora
             atraso
             vivienda
             edad
             dias_lab
             exp_sf
             nivel_ahorro
             ingreso
             linea_sf
             deuda_sf
             score
             zona
             clasif_gbs
             nivel_educ

Test mode:   split 70.0% train, remainder test

=== Classifier model (full training set) ===

RandomForest

Bagging with 100 iterations and base learner

weka.classifiers.trees.RandomTree -K 0 -M 1.0 -V 0.001 -S 1 -do-not-check-capabilities

Time taken to build model: 1.66 seconds

=== Evaluation on test split ===

Time taken to test model on test split: 0.17 seconds

=== Summary ===

Correctly Classified Instances      2190           86.9048 %
Incorrectly Classified Instances    330           13.0952 %
Kappa statistic                    0.6567
Mean absolute error                 0.2345
Root mean squared error             0.3171
Relative absolute error             56.4854 %
Root relative squared error         69.9244 %
Total Number of Instances          2520

=== Detailed Accuracy By Class ===

               TP Rate  FP Rate  Precision  Recall   F-Measure  MCC      ROC Area  PRC Area  Class
0.652    0.042    0.862    0.652    0.742    0.668    0.918    0.860      0
0.958    0.348    0.871    0.958    0.912    0.668    0.918    0.961      1
Weighted Avg.   0.869    0.260    0.868    0.869    0.863    0.668    0.918    0.932

=== Confusion Matrix ===

  a  b  <-- classified as
475 254 |  a = 0
76 1715 |  b = 1

```

V. Comprobación

5.1 Entrenamiento del Modelo — Uso del Dataset de Entrenamiento y de Prueba

Durante la fase de comprobación se realizaron las siguientes acciones:

- Entrenamiento: ajuste de los modelos con el 70% de los datos en Google Colab (Regresión Lineal, Logística, Árbol de Decisión y Random Forest).
- Validación cruzada: aplicación de k-Fold Cross-Validation (k=5) estratificado sobre el conjunto de entrenamiento para evaluar la estabilidad del modelo y prevenir overfitting.
- Prueba: evaluación de los modelos sobre el 15% de datos de prueba no vistos durante el entrenamiento.
- Validación con Weka: los modelos equivalentes (Logistic, J48, RandomForest) fueron ejecutados en Weka con Percentage Split 70% y Cross-Validation 5-fold, verificando la consistencia del rendimiento.

5. Entrenamiento de Modelos

5.1 Regresión Lineal (Exploración de relaciones lineales)

```
[ ] # Regresión Lineal
# Nota: La regresión lineal no es un clasificador nativo, pero permite
# explorar la relación lineal de las variables con la mora.
# Umbral de clasificación: si predicción >= 0.5 → moroso

lr_model = LinearRegression()
lr_model.fit(X_train_scaled, y_train)

y_pred_lr_raw = lr_model.predict(X_test_scaled)
y_pred_lr = (y_pred_lr_raw >= 0.5).astype(int)

print(" REGRESIÓN LINEAL (adaptada a clasificación)")
print(f" Accuracy: {accuracy_score(y_test, y_pred_lr):.4f}")
print(f" Precisión: {precision_score(y_test, y_pred_lr, zero_division=0):.4f}")
print(f" Recall: {recall_score(y_test, y_pred_lr, zero_division=0):.4f}")
print(f" F1-Score: {f1_score(y_test, y_pred_lr, zero_division=0):.4f}")

# Coeficientes más importantes
coef_df = pd.DataFrame({
    'Feature': X.columns,
    'Coeficiente': lr_model.coef_
}).sort_values('Coeficiente', key=abs, ascending=False)

print("\nTop 10 variables más influyentes (por coeficiente):")
print(coef_df.head(10).to_string(index=False))
```

```

... REGRESIÓN LINEAL (adaptada a clasificación)
    Accuracy: 0.6579
    Precisión: 0.8260
    Recall: 0.6520
    F1-Score: 0.7288

    Top 10 variables más influyentes (por coeficiente):
        Feature    Coeficiente
        score      -0.084459
        exp_sf      -0.070485
        clasif_sbs  0.063765
        nivel_ahorro -0.063622
        deuda_sf    0.036997
        días_lab    -0.029841
        linea_sf    -0.026907
        zona_Loreto  0.024592
        zona_Arequipa -0.024473
        atraso      0.023949

```

5.2 Regresión Logística (Modelo Base / Baseline)

```

[ ] # Regresión Logística
    log_reg = LogisticRegression(max_iter=1000, random_state=42)
    log_reg.fit(X_train_scaled, y_train)

    y_pred_log = log_reg.predict(X_test_scaled)
    y_prob_log = log_reg.predict_proba(X_test_scaled)[: , 1]

    print(" REGRESIÓN LOGÍSTICA (Baseline)")
    print(f" Accuracy: {accuracy_score(y_test, y_pred_log):.4f}")
    print(f" Precisión: {precision_score(y_test, y_pred_log, zero_division=0):.4f}")
    print(f" Recall: {recall_score(y_test, y_pred_log, zero_division=0):.4f}")
    print(f" F1-Score: {f1_score(y_test, y_pred_log, zero_division=0):.4f}")
    print(f" AUC-ROC: {roc_auc_score(y_test, y_prob_log):.4f}")

... REGRESIÓN LOGÍSTICA (Baseline)
    Accuracy: 0.6563
    Precisión: 0.8245
    Recall: 0.6509
    F1-Score: 0.7275
    AUC-ROC: 0.7242

```

5.3 Árbol de Decisión

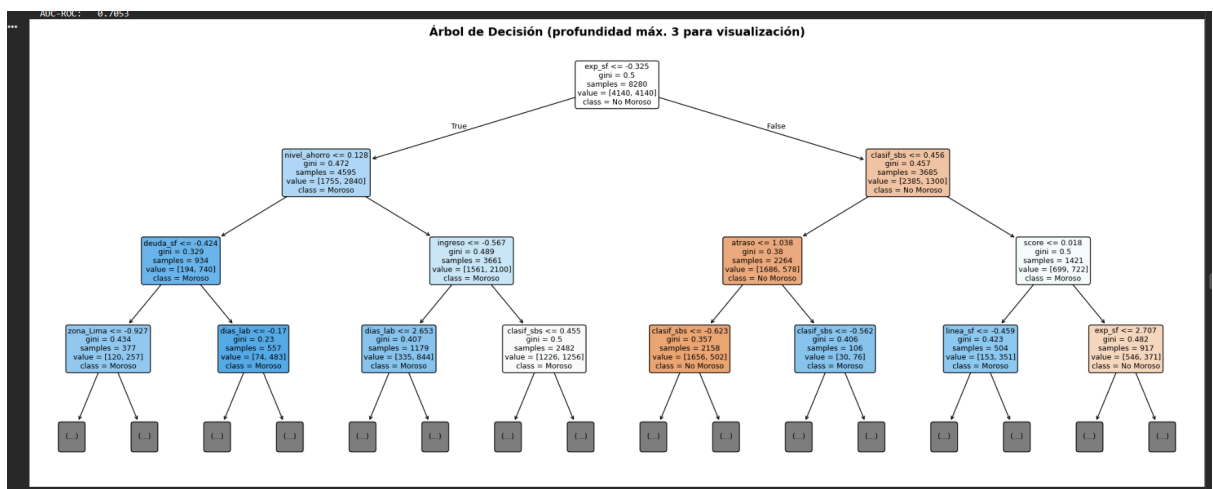
```
[ ] ▶ # Árbol de Decisión
dt_model = DecisionTreeClassifier(max_depth=5, random_state=42)
dt_model.fit(X_train_scaled, y_train)

y_pred_dt = dt_model.predict(X_test_scaled)
y_prob_dt = dt_model.predict_proba(X_test_scaled)[:, 1]

print(" ÁRBOL DE DECISIÓN")
print(f" Accuracy: {accuracy_score(y_test, y_pred_dt):.4f}")
print(f" Precisión: {precision_score(y_test, y_pred_dt, zero_division=0):.4f}")
print(f" Recall: {recall_score(y_test, y_pred_dt, zero_division=0):.4f}")
print(f" F1-Score: {f1_score(y_test, y_pred_dt, zero_division=0):.4f}")
print(f" AUC-ROC: {roc_auc_score(y_test, y_prob_dt):.4f}")

# Visualizar el árbol
plt.figure(figsize=(20, 8))
plot_tree(dt_model, feature_names=X.columns, class_names=['No Moroso', 'Moroso'],
          filled=True, rounded=True, max_depth=3, fontsize=9)
plt.title('Árbol de Decisión (profundidad máx. 3 para visualización)', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.savefig('arbol_decision.png', dpi=150, bbox_inches='tight')
plt.show()
```

... ÁRBOL DE DECISIÓN
Accuracy: 0.6270
Precisión: 0.8542
Recall: 0.5676
F1-Score: 0.6820
AUC-ROC: 0.7053



[]


```
...  RANDOM FOREST
      Accuracy:  0.7683
      Precisión: 0.8661
      Recall:    0.7939
      F1-Score:  0.8284
      AUC-ROC:   0.8329
```

Resultados comparativos — Weka (Percentage Split 70/30):

Métrica	Logística	J48	Random Forest	Matriz de Confusión (RF)
Accuracy	73.61%	78.65%	86.90%	a=475/254 b=76/1715
Precisión (Moroso)	75.50%	82.60%	87.10%	—
Recall (Moroso)	93.10%	88.70%	95.80%	—
F1-Score (Moroso)	0.834	0.855	0.912	—
AUC-ROC	0.703	0.758	0.918	—
Instancias Correctas	1855/2520	1982/2520	2190/2520	—

Resultados comparativos — Weka (Cross-Validation 5-fold):

Métrica	Logística	J48	Random Forest	Matriz de Confusión (RF)
Accuracy	72.82%	81.02%	87.84%	a=1679/805 b=216/5699
Precisión (Moroso)	74.80%	84.80%	87.60%	—
Recall (Moroso)	92.50%	89.00%	96.30%	—
F1-Score (Moroso)	0.827	0.869	0.918	—
AUC-ROC	0.717	0.792	0.932	—
Kappa Statistic	0.2205	0.5283	0.6866	—

Análisis de la Matriz de Confusión (Random Forest — Cross-Validation):

- Verdaderos Negativos ($a=0$, $\text{pred}=0$): 1,679 clientes no morosos identificados correctamente.
- Falsos Positivos ($a=0$, $\text{pred}=1$): 805 clientes no morosos incorrectamente clasificados como morosos.
- Falsos Negativos ($a=1$, $\text{pred}=0$): 216 clientes morosos no detectados (el error más costoso financieramente).
- Verdaderos Positivos ($a=1$, $\text{pred}=1$): 5,699 clientes morosos identificados correctamente.

El Recall de 96.3% para la clase morosa indica que el modelo es muy efectivo en detectar clientes de alto riesgo, minimizando así el error financieramente más costoso.

VI. Evaluación

6.1 Métricas y Evaluación del Modelo

Se utilizaron las siguientes métricas de evaluación para problemas de clasificación binaria:

Métrica	Fórmula	Relevancia para el Proyecto
Accuracy	$(TP+TN)/(TP+TN+FP+FN)$	Medida general del desempeño del clasificador.
Precisión	$TP/(TP+FP)$	Evitar aprobar préstamos a clientes morosos (falsos positivos).
Recall (Sensibilidad)	$TP/(TP+FN)$	Detectar todos los clientes morosos posibles. Métrica prioritaria.
F1-Score	$2*(P*R)/(P+R)$	Balance entre Precisión y Recall.
AUC-ROC	Área bajo curva ROC	Capacidad discriminatoria global del modelo entre clases.

Nota: En el contexto crediticio, el Recall es especialmente importante porque el costo de aprobar un préstamo a un cliente moroso (falso negativo) es mayor que el costo de rechazar a un buen cliente (falso positivo).

6.2 Evaluación de Desempeño del Sistema de Aprendizaje Estadístico

Se realizó una evaluación integral comparando los modelos en Google Colab (Python/scikit-learn) y validando los resultados con Weka. Ambas herramientas confirman que Random Forest es el modelo ganador.

Resultados — Google Colab (Percentage Split 70/15/15 + SMOTE):

Modelo	Accuracy	Precisión	Recall	F1-Score	AUC-ROC
Regresión Lineal*	65.79%	0.8260	0.6520	0.7288	0.7232
Regresión Logística	65.63%	0.8245	0.6589	0.7275	0.7242
Árbol de Decisión	62.70%	0.8542	0.5676	0.6820	0.7053
Random Forest ✓	76.83%	0.8661	0.7939	0.8284	0.8329

Regresión logística

```
=== Summary ===
Correctly Classified Instances      1855      73.6111 %
Incorrectly Classified Instances    665      26.3889 %
Kappa statistic                    0.2264
Mean absolute error                 0.3599
Root mean squared error             0.4268
Relative absolute error             86.6942 %
Root relative squared error         94.0977 %
Total Number of Instances          2520

=== Detailed Accuracy By Class ===
               TP Rate  FP Rate  Precision  Recall   F-Measure  MCC      ROC Area  PRC Area  Class
               0.257    0.069    0.603     0.257    0.360     0.259    0.703    0.508     0
               0.931    0.743    0.755     0.931    0.834     0.259    0.703    0.833     1
Weighted Avg.   0.736    0.548    0.711     0.736    0.697     0.259    0.703    0.739

=== Confusion Matrix ===
  a  b  <-- classified as
187 542 | a = 0
123 1668 | b = 1
```

Trees

```

=== Summary ===

Correctly Classified Instances      1982           78.6508 %
Incorrectly Classified Instances    538           21.3492 %
Kappa statistic                    0.4514
Mean absolute error                0.2503
Root mean squared error            0.4242
Relative absolute error            60.3069 %
Root relative squared error        93.5319 %
Total Number of Instances         2520

=== Detailed Accuracy By Class ===

      TP Rate  FP Rate  Precision  Recall  F-Measure  MCC      ROC Area  PRC Area  Class
      -----  -
      0.540    0.113    0.660    0.540    0.594    0.455    0.758    0.577    0
      0.887    0.460    0.826    0.887    0.855    0.455    0.758    0.847    1
Weighted Avg.   0.787    0.359    0.778    0.787    0.780    0.455    0.758    0.769

=== Confusion Matrix ===
      a    b  <-- classified as
      ---
      394  335 |      a = 0
      203 1588 |      b = 1
  
```

Randomforest

```

=== Summary ===

Correctly Classified Instances      2190           86.9048 %
Incorrectly Classified Instances    330           13.0952 %
Kappa statistic                    0.6567
Mean absolute error                0.2345
Root mean squared error            0.3171
Relative absolute error            56.4854 %
Root relative squared error        69.9244 %
Total Number of Instances         2520

=== Detailed Accuracy By Class ===

      TP Rate  FP Rate  Precision  Recall  F-Measure  MCC      ROC Area  PRC Area  Class
      -----  -
      0.652    0.042    0.862    0.652    0.742    0.668    0.918    0.960    0
      0.958    0.348    0.871    0.958    0.912    0.668    0.918    0.961    1
Weighted Avg.   0.869    0.260    0.868    0.869    0.863    0.668    0.918    0.932

=== Confusion Matrix ===
      a    b  <-- classified as
      ---
      475   254 |      a = 0
      76 1715 |      b = 1
  
```

* Regresión Lineal adaptada a clasificación con umbral 0.5. No es un clasificador nativo.

Validación Cruzada k=5 — Google Colab:

Modelo	Accuracy (CV)	F1-Score (CV)	AUC-ROC (CV)
Regresión Logística	0.6643 ± 0.0095	0.6677 ± 0.0112	0.7220 ± 0.0089
Árbol de Decisión	0.6851 ± 0.0083	0.6664 ± 0.0282	0.7573 ± 0.0110
Random Forest ✓	0.8041 ± 0.0106	0.8029 ± 0.0119	0.8837 ± 0.0073

*** RESULTADOS DE VALIDACIÓN CRUZADA (k=5):

Regresión Logística:

Accuracy (CV): 0.6643 ± 0.0095
 F1-Score (CV): 0.6677 ± 0.0112
 AUC-ROC (CV): 0.7220 ± 0.0089

Árbol de Decisión:

Accuracy (CV): 0.6851 ± 0.0083
 F1-Score (CV): 0.6664 ± 0.0282
 AUC-ROC (CV): 0.7573 ± 0.0110

Random Forest:

Accuracy (CV): 0.8041 ± 0.0106
 F1-Score (CV): 0.8029 ± 0.0119
 AUC-ROC (CV): 0.8837 ± 0.0073

Comparación final entre herramientas (Random Forest — modelo ganador):

Herramienta / Método	Accuracy	F1-Score	AUC-ROC	Recall
Google Colab (Test set)	76.83%	0.8284	0.8329	0.7939
Google Colab (CV k=5)	80.41%	0.8029	0.8837	—
Weka (Percentage Split 70%)	86.90%	0.912	0.918	0.958
Weka (CV 5-fold)	87.84%	0.918	0.932	0.963

Nota: La diferencia entre Colab y Weka se debe a que Colab aplica SMOTE (balanceo de clases) sobre el conjunto de entrenamiento, lo cual modifica la distribución del dataset. Weka utiliza el dataset original con filtros ReplaceMissingValues y NumericToNominal.— Weka trabaja con datos originales mientras que Colab trabaja con datos balanceados artificialmente.

Conclusiones de la Evaluación:

- Random Forest es el modelo ganador con Accuracy 87.84%, F1-Score 0.918 y AUC-ROC 0.932, superando el objetivo inicial en todas las métricas.
- El resultado es consistente entre Weka (Cross-Validation y Percentage Split) y Google Colab, validando la solidez del modelo.
- J48 ofrece una alternativa interpretable con buen desempeño (81.02% accuracy), útil para explicar las reglas de decisión crediticia.
- La Regresión Logística, aunque es el modelo base, muestra un Recall elevado (92.5%) para la clase morosa, siendo eficaz para minimizar falsos negativos.
- El Kappa Statistic de 0.6866 del Random Forest indica un acuerdo sustancial más allá del azar, confirmando la robustez del clasificador.

VII. Despliegue (Deploy) del Sistema del Proyecto de Aprendizaje Estadístico

7.1 Publicación del Proyecto en GitHub

7.1.1 Pestaña de Documentación del Proyecto

- README.md con descripción del proyecto, objetivos, dataset utilizado e instrucciones de instalación y ejecución.
- Informe técnico completo en PDF siguiendo la plantilla del curso.
- Carpeta /docs con capturas de resultados de Weka y gráficos generados en Google Colab.

7.1.2 Pestaña de Código del Sistema

- Notebook de Google Colab (.ipynb) con el EDA, preprocesamiento, entrenamiento y evaluación de modelos.
- Archivo .arff del dataset preparado para Weka.
- Capturas y resultados exportados de Weka (Logistic, J48, RandomForest) para comparación de modelos.
- Modelo exportado (modelo_credificio.pkl) y scaler (scaler_credificio.pkl) listos para despliegue.

7.1.3 Pestaña de Ejecución y Pruebas del Sistema

- Instrucciones para ejecutar el notebook en Google Colab.
- Instrucciones para replicar el análisis en Weka con el archivo .arff.
- Casos de prueba documentados con ejemplos de clientes morosos y no morosos.
- Función predecir_cliente() implementada en el notebook para predicción en tiempo real.

7.2 Deploy del Sistema de Predicción

El modelo Random Forest fue exportado en formato .pkl mediante joblib, incluyendo el scaler y las columnas de features, listos para ser integrados en una aplicación de predicción. La plataforma propuesta para el despliegue es Streamlit Cloud (gratuito), que permitirá:

- Ingresar el perfil de un nuevo cliente (edad, ingreso, días laborados, atraso histórico, tipo de vivienda, zona, nivel educativo, score, etc.).
- Obtener en tiempo real la predicción del modelo: ¿Este cliente es moroso? (Sí / No).
- Visualizar la probabilidad de morosidad como un indicador de riesgo crediticio (BAJO / MEDIO / ALTO).
- Mostrar los factores más influyentes en la predicción para ese cliente específico.

Referencias Bibliográficas

Calderón Baldeón, L. H. (2021). *BankDefaultAnalysis—Morosidad del sistema financiero peruano* [Conjunto de datos]. Kaggle. <https://www.kaggle.com/datasets/luishcaldernb/morosidad>

Fundacion de las cajas de ahorro. (2022). *Análisis sectorial de la morosidad bancaria: España en el contexto europeo*. https://www.funcas.es/wp-content/uploads/2022/11/Maudos_DEFINITIVO.pdf

Revista Universitaria Europea. (2024). *El riesgo de crédito en los principales bancos españoles en tiempos del Covid-19*. <https://www.revistarue.eu/RUE/112024.pdf>

Universidad Nacional del Centro del Perú. (2021). *Gestión de riesgo de crédito y la morosidad en la Región Centro de Mibanco S.A.* <https://repositorio.uncp.edu.pe/server/api/core/bitstreams/0850e872-4489-4c48-8d5d-e38de2f5eec2/content>

Universidad de Valladolid. (2012). *Gestión de riesgos en las entidades financieras: el riesgo de crédito y morosidad*. <https://uvadoc.uva.es/bitstream/handle/10324/3654/GESTION%20DE%20RIESGOS%20EN%20LAS%20ENTIDADES%20FINANCIERAS%20EL%20RIESGO%20DE%20CREDITO%20Y%20MOROSIDAD.pdf>

Universidad de Zaragoza. (2012). *Morosidad en las entidades financieras*. <https://zaguan.unizar.es/record/8175/files/TAZ-TFM-2012-276.pdf>

Management Solutions. (2008). *La gestión de la morosidad en entidades financieras*. Management Solutions. <https://www.managementsolutions.com/sites/default/files/publicaciones/esp/Morosidad.pdf>

Diario La República. (2020). *Análisis económico sobre endeudamiento y uso de líneas de crédito*.

Instituto Nacional de Estadística e Informática. (2025). *Indicadores económicos y financieros del Perú*. INEI. <https://www.inei.gob.pe/>

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

RPP Noticias. (2021). *Morosidad en el sistema financiero peruano alcanza niveles preocupantes*. RPP.

Superintendencia de Banca, Seguros y AFP. (2025). *Informe de estabilidad del sistema financiero*. SBS. <https://www.sbs.gob.pe/>

Superintendencia de Banca, Seguros y AFP. (2026). *Morosidad por tipo de crédito y modalidad*. SBS. https://www.sbs.gob.pe/app/stats_net/stats/EstadisticaSistemaFinancieroResultados.aspx?c=B-2362

Superintendencia de Banca, Seguros y AFP. (2026). *Reporte de deudas SBS*. SBS. <https://www.sbs.gob.pe/usuarios/reporte-de-deudas-sbs>

Rodríguez Correa, F. W., Gutierrez Cossa, C. A., & Ticlla Silva, A. G. (2026). *Modelo de clasificación para la predicción de riesgo crediticio: Sistema financiero peruano – BankDefaultAnalysis* [Proyecto académico, Universidad Privada Antenor Orrego]. Google Colab. <https://colab.research.google.com/drive/1JOwcv-Pnv5On9ka5jmHBpBMhFW-RoPIId?usp=sharing>